

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

РАСЧЕТНАЯ РАБОТА
по дисциплине «Представление и обработка информации в интеллектуальных системах»
на тему
Проверить граф на двусвязность

Выполнил:

Ю . С. Ануфриев

Студент группы
421701

Проверил:

Н. В. Малиновская

Минск 2025

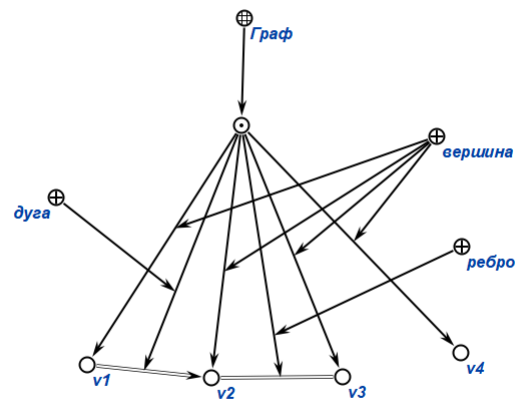
1 Список понятий

Введение :

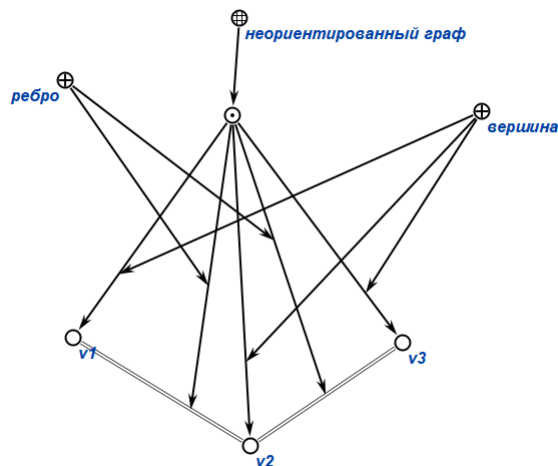
Цель: Изучить основы теории графов, способы представления графов, базовые алгоритмы для работы с графами. Получить навыки формализации и обработки информации с использованием семантических сетей.

Задача: Проверить граф на двусвязность.

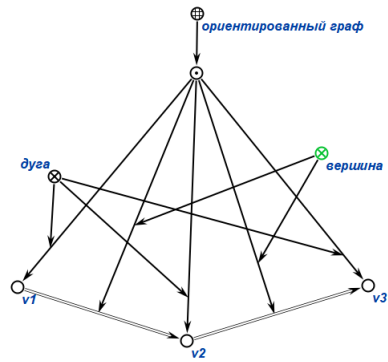
1. **Граф** - математическая абстракция реальной системы любой природы, объекты которой обладают парными связями. (совокупность точек, соединенных линиями. Точки называются вершинами, или узлами, а линии – ребрами, или дугами.) :



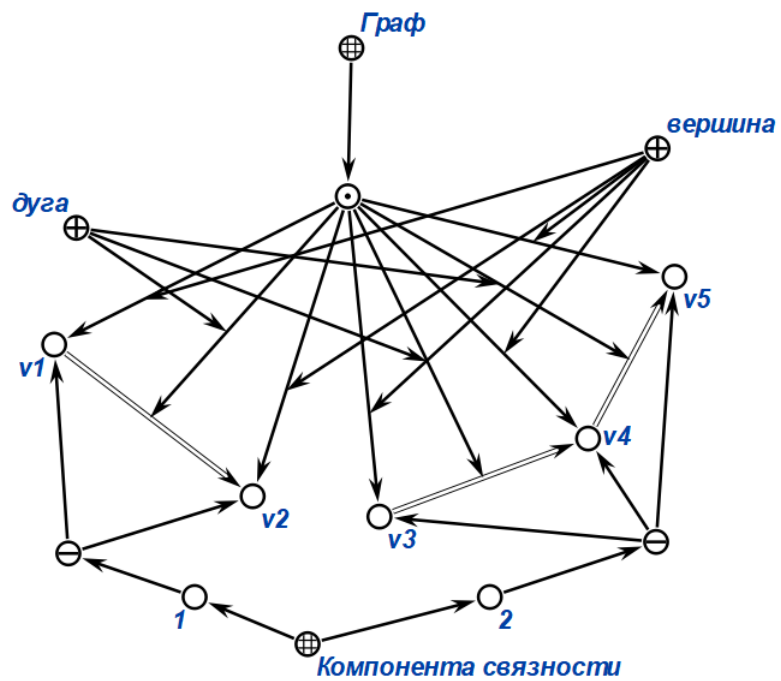
2. **Неориентированный граф** — Граф, ни одному ребру которого не присвоено направление.



3. *Ориентированный граф* — граф, рёбрам которого присвоено направление.



4. *Компонента связности* — множество всех связанных вершин графа.

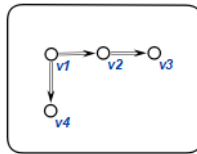


2 Примеры графов

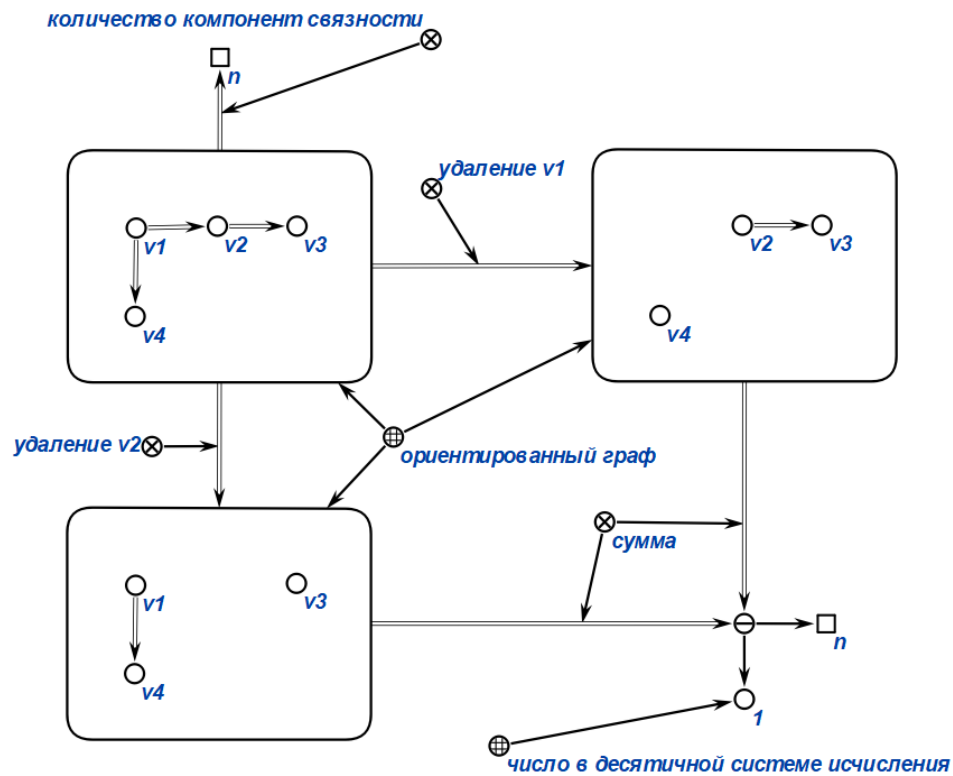
2.1 Пример 1

Вход:

Необходимо найти множество вершин, удаление которых приводит к увеличению числа компонент связности графа. Переменная n хранит число компонент связности входного графа.

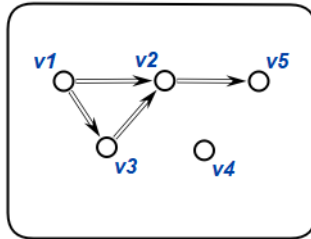


Выход: Удаление вершин $v1$ и $v2$ приводит к увеличению числа компонент связности хранящихся в переменной n .

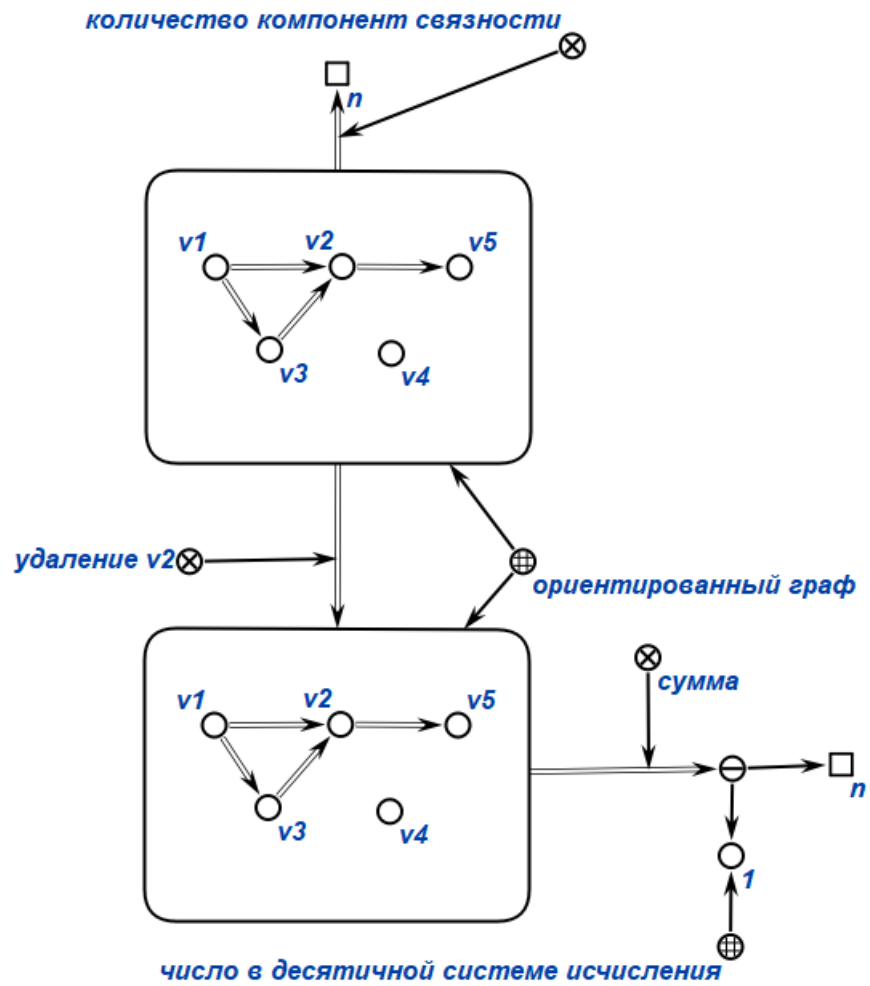


2.2 Пример 2

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

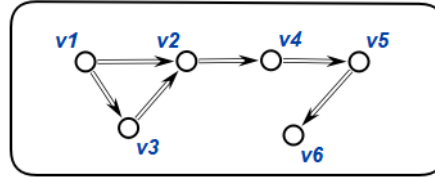


Выход: Вершина v2

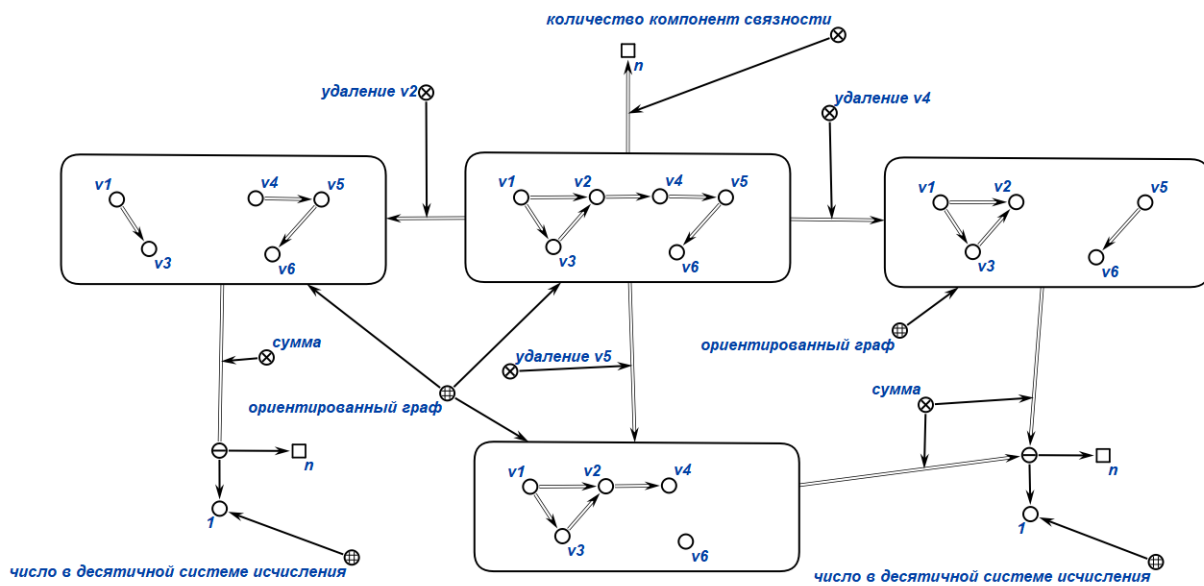


2.3 Пример 3

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

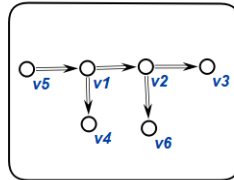


Выход: Вершины v2, v4, v5

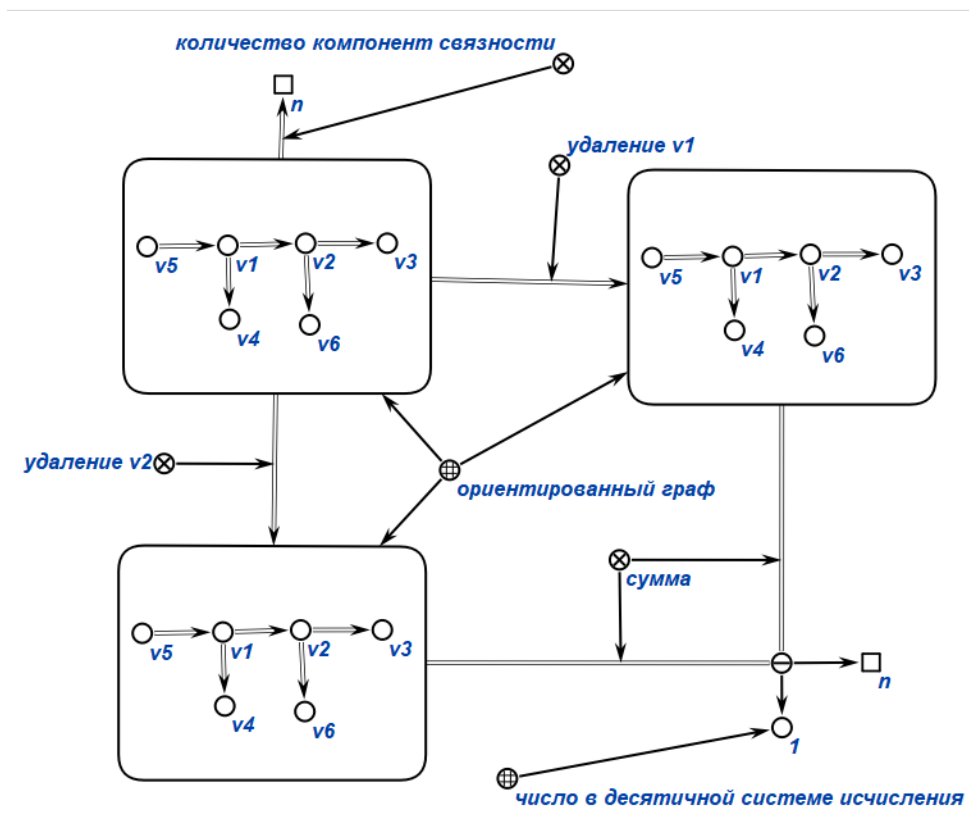


2.4 Пример 4

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

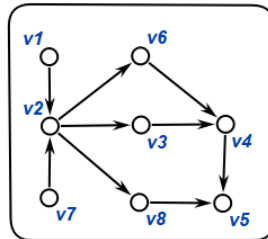


Выход: Вершины v_1, v_2

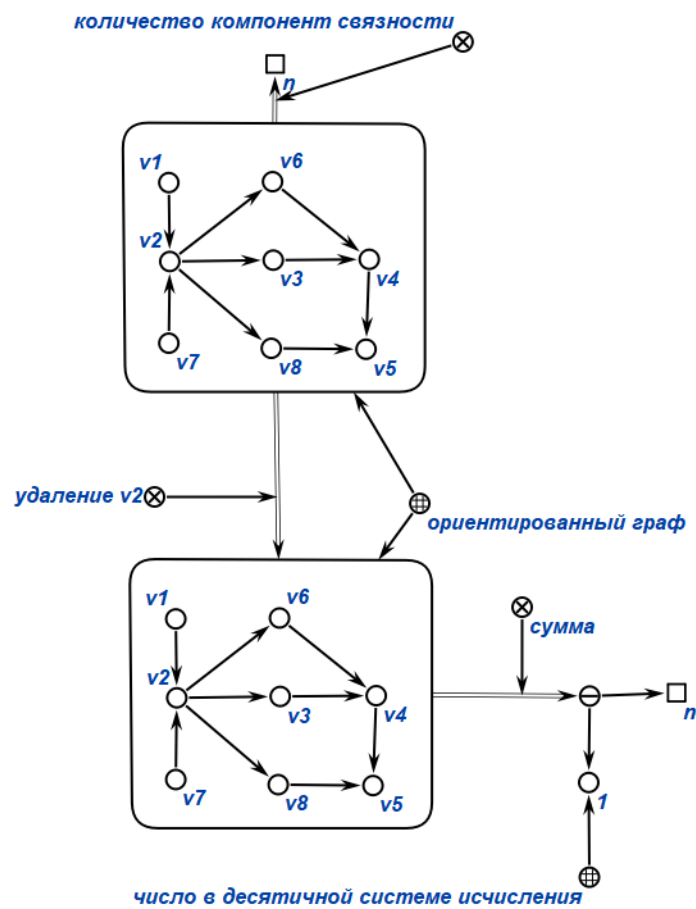


2.5 Пример 5

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

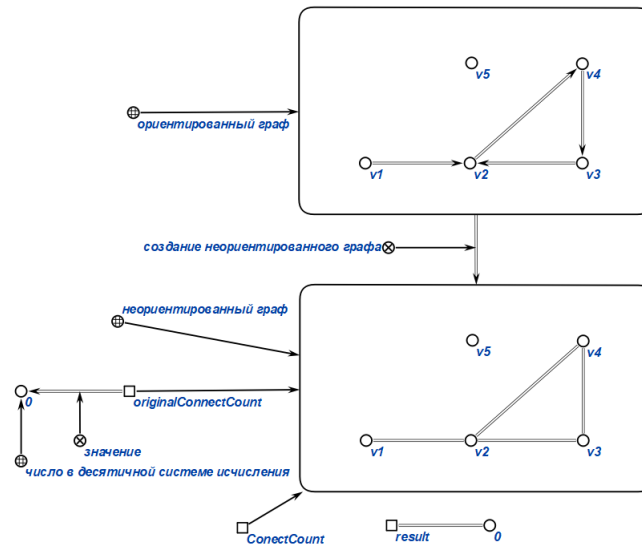


Выход: Вершина v2

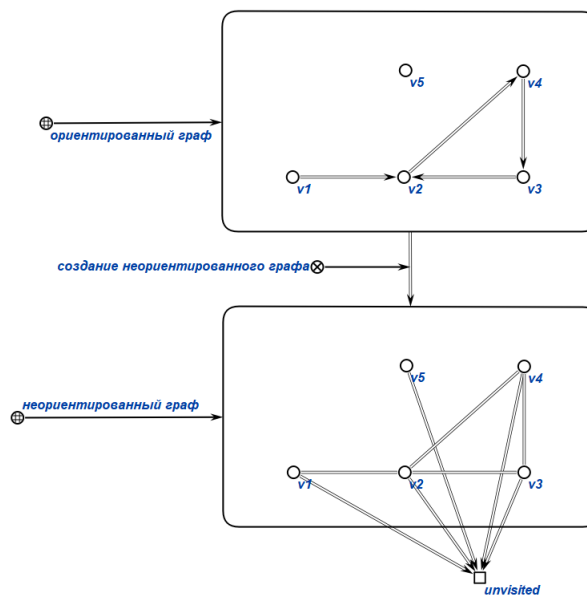


3 Алгоритм решения теоретико-графической задачи

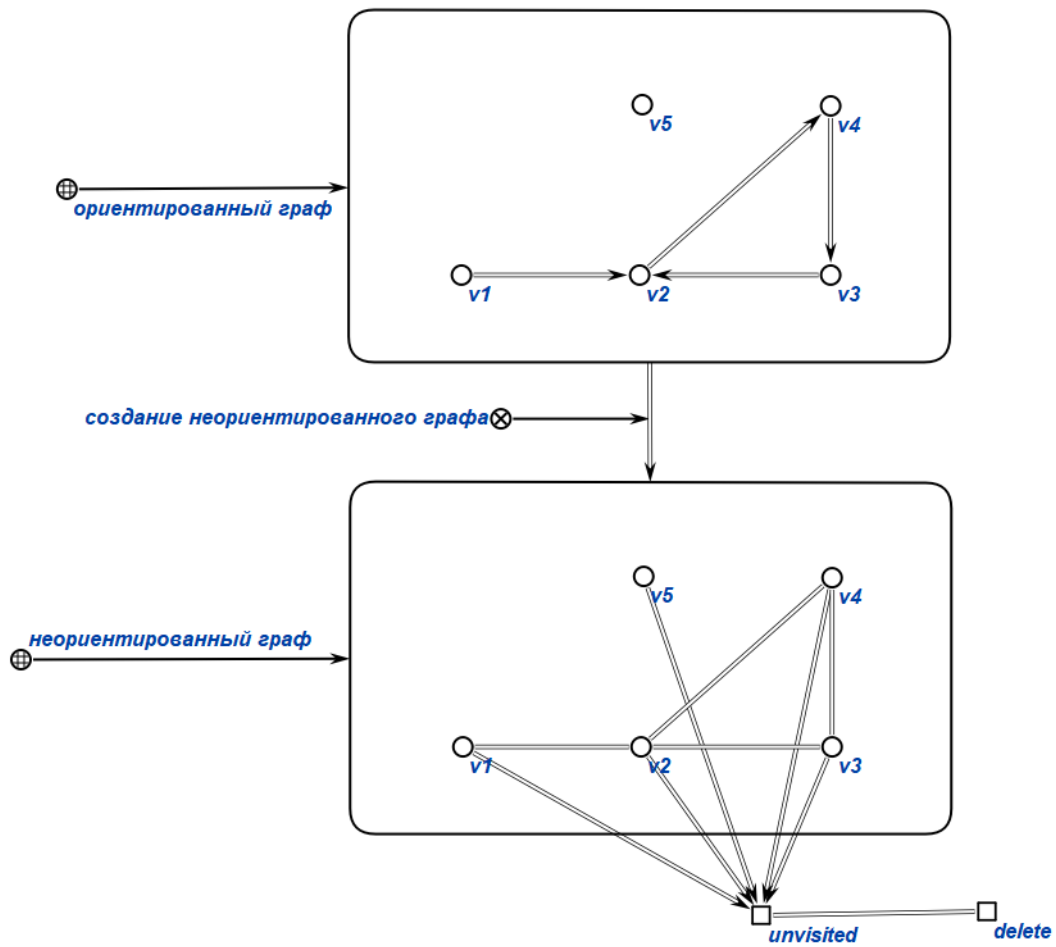
1. Получаем в качестве входного графа пользовательский граф. Создаём неориентированный граф из входного.
2. Создаём переменную `originalConnectCount` которая будет хранить количество компонентов связности во входном графе, счетчик `ConnectCount` который будет считать количество компонентов связности в каждом графе и `result` хранящий результат.



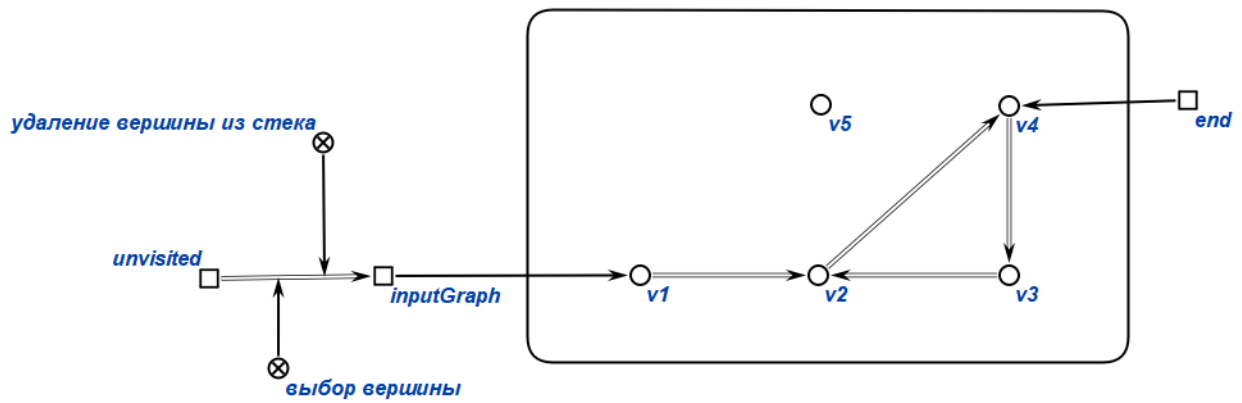
3. Создаём переменную `unvisited` - стек из непосещённых вершин.



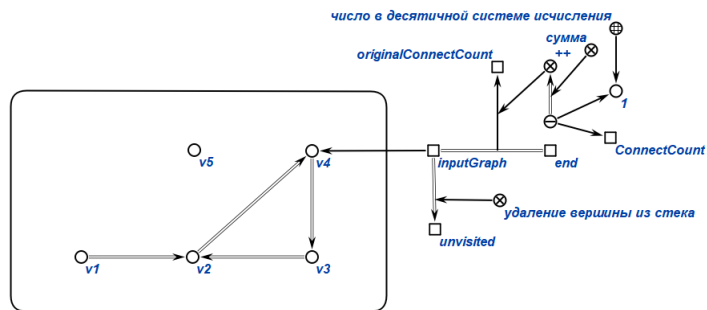
4. Создаём стек delete который будет поочередно удалять каждую вершину входного графа



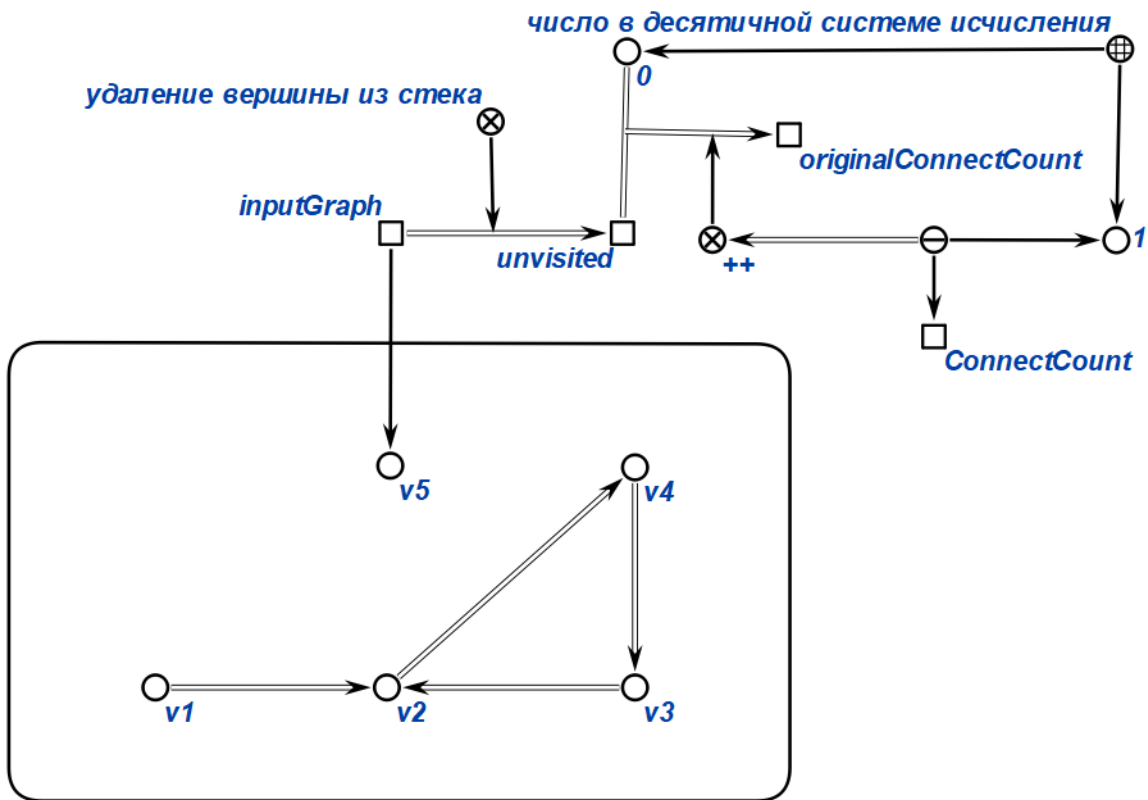
5. Выбираем и удаляем из стека первую вершину inputGraph; end показывает последнюю вершину до которой есть связанные с нашей вершиной рёбра.



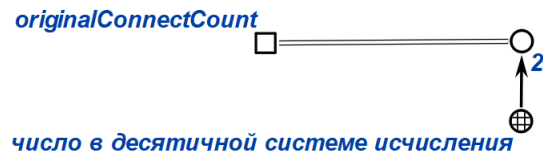
6. После достижения конечной вершины количество компонентов связности ConnectCount увеличивается на 1 (изначально в переменной хранится 0); А также каждая посещённая вершина удаляется из стека.



7. При нахождении отдельно стоящей вершины количество компонентов связности также увеличивается на 1.
8. При достижении стеком unvisited нуля количество компонентов связности из ConnectCount переходит в переменную originalConnectCount. Это означает что теперь входной граф содержит данное количество компонентов связности.

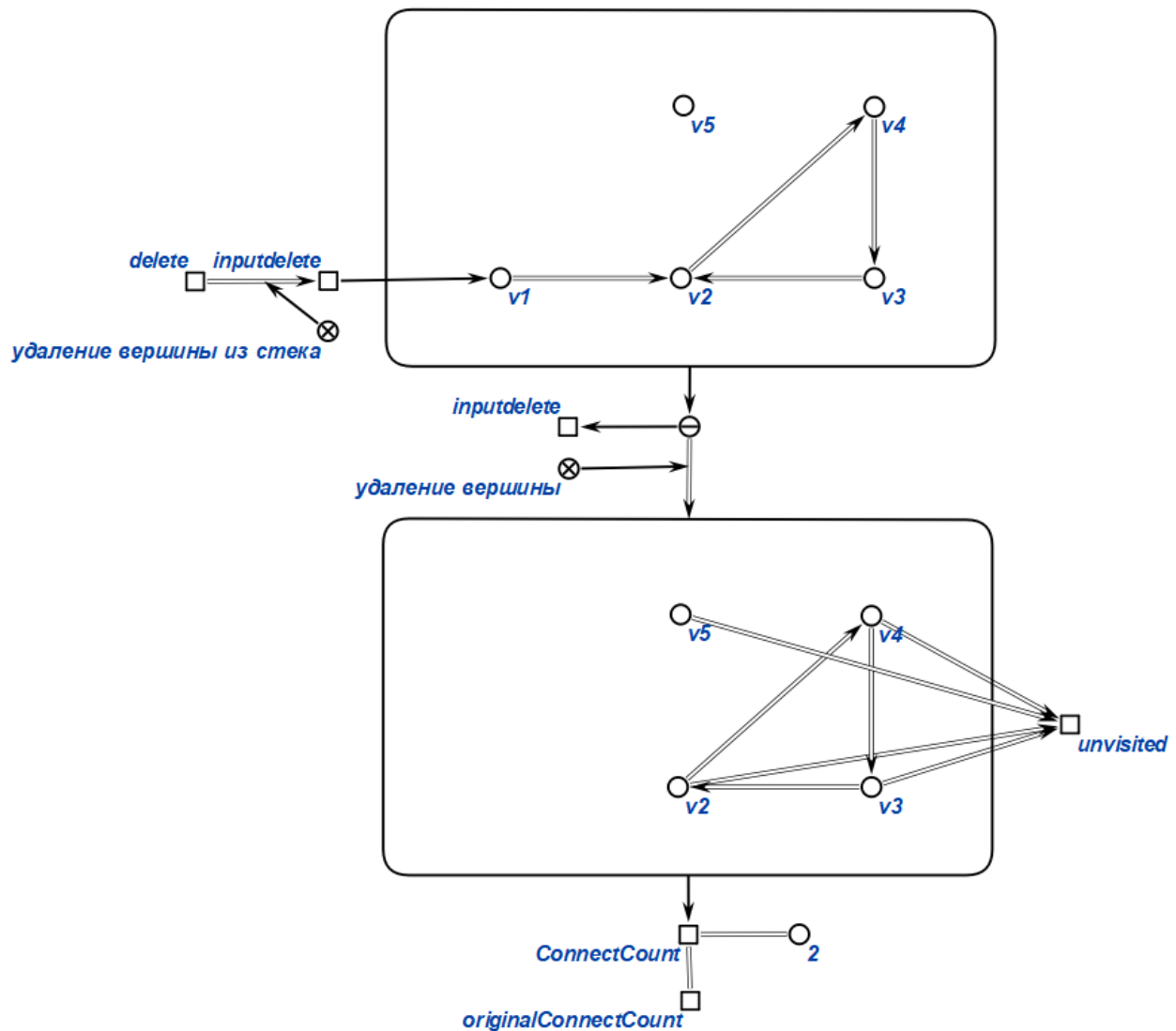


9. Количество компонент связности в взвешенном графе равно 2.

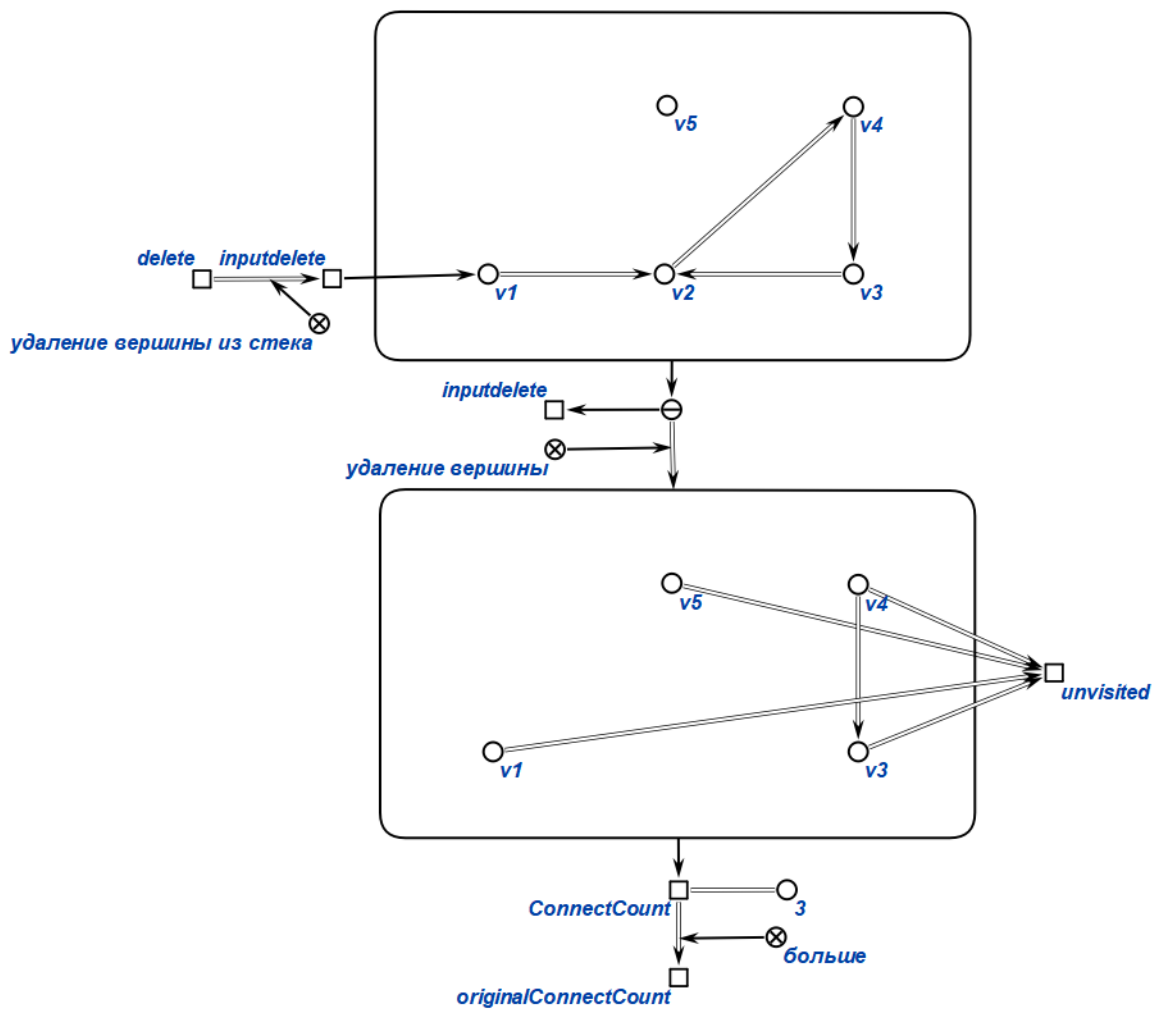


10. Выбираем первую вершину из стека delete и удаляем ее из входного графа для проверки на количество компонент связности.

11. С полученным графом повторяем те же действия что и с входным. Полученное значение ConectCount сравниваем с originalConectCount. До тех пор пока количество компонент связности данного графа не больше чем у входного - переходим к последующей вершине из delete.



12. Повторяем алгоритм пока стек delete не опустеет. Если значение в каком-либо графе окажется больше чем во входном - то эта вершина добавляется в result для дальнейшего вывода результата



13. Вывод хранящейся в переменной result информации как результат выполнения задачи.



4 Выводы

Получил навыки формализации и обработки информации с использованием семантических сетей на примере работы алгоритма конкретной задачи.

5 Литература

- [1] Емеличев В. А., Мельников О. И., Сарванов В. И., Тышкевич Р. И. Лекции по теории графов. М.: Наука, 1990. 384с. (Изд.2, испр. М.: УРСС, 2009. 392 с.)
- [2] Оре О. Теория графов. – 2-е изд.. – М.: Наука, 1980. – С. 336.
- [3] <https://www.youtube.com/watch?v=Z4tAyq8txDg>
- [4] <https://www.youtube.com/watch?v=iQabtd6VBL0>
- [5] <https://www.youtube.com/watch?v=3-XLRh2M5YI>